

Reviewer Pack — Protocol-Induced Covers on Tensor-Powered Girth Gadgets Exhi...

Entry 578b4afa80c0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 07:23:26 UTC

Protocol-Induced Covers on Tensor-Powered Girth Gadgets Exhibit Logarithmic Multiplicity Growth

Entry ID: 578b4afa80c0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 07:23:26 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Geometric Complexity Theory

Field B (complexity object): Communication complexity class $CC(f \circ G^n)$

Statement:

For a MetricGadget G with positive asymptotic dimension ($\text{asdim}_R(G) \geq 1$ at scale $R = \text{diam}(G)$), and its n -fold tensor power $G^{\{\otimes k\}}$ used as a gadget in the lift $f \circ (G^{\{\otimes k\}})^n$, every deterministic communication protocol Π computing $f \circ (G^{\{\otimes k\}})^n$ with cost c induces a ProtocolPullback that forms a RoeCover of multiplicity $m_\Pi \geq \log_2(k+1)$ for sufficiently large k , where the scale R_Π depends only on G .

Rationale (proposer's reasoning):

This sub-conjecture tests Axiom A3 (Asdim Amplification) in tandem with A2 (Protocol $\rightarrow$ Cover). If $\text{asdim}(G^{\{\otimes k\}})$ grows at least linearly in k (up to constants), then A2 implies that any protocol must induce a cover whose multiplicity is at least the logarithm of the asdim-amplified structure. The ProtocolPullback must therefore reflect this coarse geometric blowup, forcing m_Π to grow with k . Since CLC is meant to bound communication complexity via coarse invariants, failure of multiplicity to grow logarithmically would invalidate the amplification mechanism needed for super-logarithmic lower bounds.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 04263c57a8ae5079

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Protocol-induced covers exhibit logarithmic multiplicity growth

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse geometric lifting AND communication complexity class-geometric complexity theory AND tensor power AND logarithmic multiplicity growth - roe cover AND protocol induced AND asymptotic dimension AND metric gadget

Top relevant hits considered: - [<http://arxiv.org/abs/1903.06084v8>] Lifting coarse homotopies - [<http://arxiv.org/abs/1803.07099v3>] Varieties of coarse spaces - [<http://arxiv.org/abs/2511.17227v3>] A Lifting Theorem for Hybrid Classical-Quantum Communication Complexity - [<http://arxiv.org/abs/2211.07055v1>] Geometric complexity theory for product-plus-power - [<http://arxiv.org/abs/cs/0608054v2>] Dispersion of Mass and the Complexity of Randomized Geometric Algorithms - [<http://arxiv.org/abs/1011.1350v1>] Geometric Complexity Theory and Tensor Rank - [<http://arxiv.org/abs/2412.16766v2>] A Protocol for KG Construction Tasks Involving Users - [<http://arxiv.org/abs/2201.06488v1>] On Hochschild homology of uniform Roe algebras with coefficients in uniform Roe bimodules

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import product

# Constants
N_MAX = 40
SEEDS = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3 # Default seeds if none provided

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def OR(x):
        return max(x)

    def GadgetComposition(G1, G2):
        n1, m1 = len(G1), len(G1[0])
        n2, m2 = len(G2), len(G2[0])
        result = [[0] * (m1 * m2) for _ in range(n1 * n2)]
        for i in range(n1):
            for j in range(m1):
                for k in range(n2):
```

```

        for l in range(m2):
            if G1[i][j] == 1 and G2[k][l] == 1:
                result[i*n2+k][j*m2+l] = 1
    return result

def lift_function(f, G):
    n = len(G)
    lifted_G = [[0] * (n**2) for _ in range(n**2)]
    for i in range(n):
        for j in range(n):
            lifted_G[i*n+j][i*n:(i+1)*n] = [f(G[i][j])]
            lifted_G[i*n+j][(i+1)*n:] = [0] * (n*(n-1))
    return lifted_G

def ProtocolPullback( $\Pi$ , G):
    n = len(G)
    pullback = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if  $\Pi[i][j]$  == 1:
                pullback[i][j] = 1
    return pullback

def CoarsePullback(pullback, G):
    n = len(G)
    coarse_pullback = [[0] * (n//2) for _ in range(n//2)]
    for i in range(0, n, 2):
        for j in range(0, n, 2):
            if pullback[i][j] == 1 or pullback[i+1][j] == 1 or pullback[i][j+1] == 1 or pullback[i+1][j+1] == 1:
                coarse_pullback[i//2][j//2] = 1
    return coarse_pullback

def asdim(G):
    n = len(G)
    m = len(G[0])
    if n == 1 and m == 1:
        return 0
    for d in range(1, min(n, m)):
        found = False
        for i in range(d, n, d):
            for j in range(d, m, d):
                if all(G[x][y] == G[x+d-1][y+d-1] for x in range(i-d+1, i) for y in range(j-d+1, j)):
                    found = True
                    break
            if found:
                break
        if not found:
            return d - 1
    return min(n, m) - 1

G = [[0, 1], [1, 0]] # Base gadget: 2-bit equality
n_values = [5, 10, 15, 20, 30, 40]

results = []
for k in range(1, N_MAX + 1):
    G_k = G

```

```

for _ in range(k-1):
    G_k = GadgetComposition(G_k, G)

for n in n_values:
    lifted_G = lift_function(OR, G_k)
    f = OR
    lifted_function = lambda x: [f(G_k[x[i]]) for i in range(n)]

    # Generate all deterministic protocols of cost c <= 5
    protocols = []
    for c in range(6):
        for protocol in product([0, 1], repeat=n*n):
            if sum(protocol) == c:
                protocols.append(protocol)

    for Π in protocols:
        pullback = ProtocolPullback(Π, lifted_G)
        coarse_pullback = CoarsePullback(pullback, G_k)
        m_Π = sum(sum(row) for row in coarse_pullback)

        if m_Π < math.log2(k+1):
            return {
                "metric_name": "Multiplicity",
                "metric_value": m_Π,
                "instances_tested": len(protocols),
                "conjecture_holds": False,
                "counterexample": f"Protocol Π with cost {sum(Π)} failed for k={k}, n={n}"
            }

    return {
        "metric_name": "Multiplicity",
        "metric_value": sum(sum(row) for row in coarse_pullback),
        "instances_tested": len(protocols),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else SEEDS

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Multiplicity did not grow logarithmically\" first failing seed={first_failing_seed}")
    else:
        print(f"RESULT: UNKNOWN support_fraction={support_fraction}")

```

```

else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_309b7e61.py", line 133, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_309b7e61.py", line 94, in run_trial
    lifted_G = lift_function(OR, G_k)
                ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_309b7e61.py", line 44, in lift_function
    lifted_G[i*n+j][i*n:(i+1)*n] = [f(G[i][j])]
                                   ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_309b7e61.py", line 25, in OR
    return max(x)
           ^^^^^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, preventing evaluation of the conjecture. | next:
Fix the test code to handle the TypeError and re-run the experiment.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	groq	llama-3.3-70b-versat	0	0	4289
2	novelty	groq	llama-3.3-70b-versat	0	0	4073
3	novelty	groq	llama-3.3-70b-versat	0	0	4646
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15144
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16129
6	judge	groq	llama-3.3-70b-versat	0	0	4579

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 48860 ms total latency. Provider mix: {'groq': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/578b4afa80c0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/578b4afa80c0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/578b4afa80c0.tar.gz` (if generated)